Word-Counter:
다양한 문서 형식에서 단어 집계

분석 · 조회 · 저장 · 성능 개선

발표자: 박상호
발표일: 2026-10-10

처리 흐름

01
입력 경로 확인
파일 또는 디렉터리 탐색

02
TextReader 선택
확장자별 본문 추출

03
Tokenizer 토큰화
텍스트를 단어로 분리, 정규화

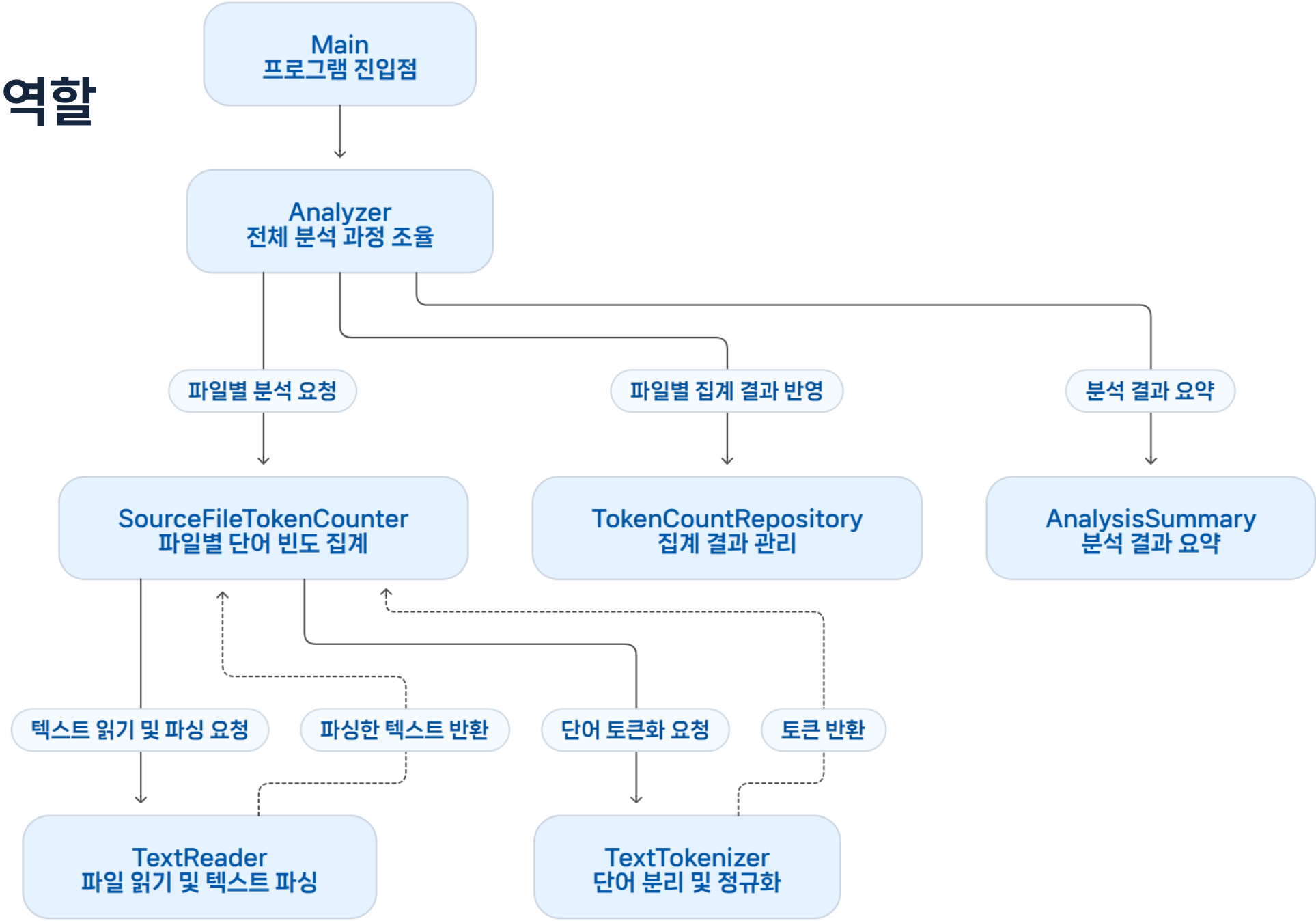
04
파일별 빈도 집계
Map으로 단어별 횟수 집계

05
전체 결과 합산
처리에 성공한 결과만 반영

06
결과 조회·저장
상위·특정 단어 조회와 TSV 파일 저장

파일 읽기와 단어 토큰화를 분리하고, 파일별 집계 결과를 전체 결과로 합산

주요 클래스와 역할


Main
프로그램 진입점
Analyzer
전체 분석 과정 조율
파일별 분석 요청
파일별 집계 결과 반영
분석 결과 요약
SourceFileTokenCounter
파일별 단어 빈도 집계
TokenCountRepository
집계 결과 관리
AnalysisSummary
분석 결과 요약
텍스트 읽기 및 파싱 요청
파싱한 텍스트 반환
단어 토큰화 요청
토큰 반환
TextReader
파일 읽기 및 텍스트 파싱
TextTokenizer
단어 분리 및 정규화

기본 기능 시연

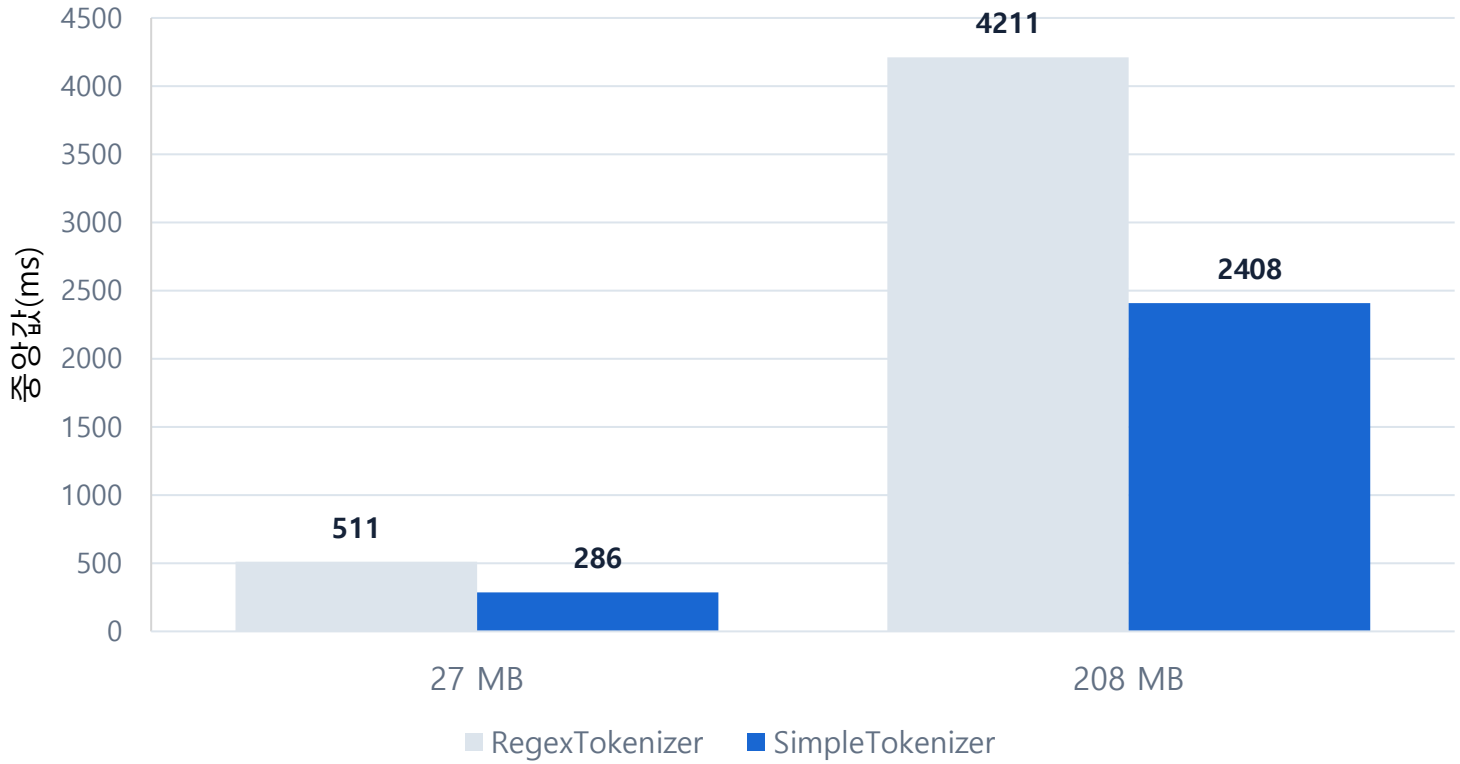
다양한 파일 형식의 단어 집계부터 결과 조회·저장, 오류 처리까지 확인

1	파일 분석	samples/equivalent
2	결과 조회	news-1000.csv
3	결과 저장	out/counts.tsv
4	오류 처리	samples/invalid

정상 파일의 결과는 유지하고, 실패 파일의 부분 집계는 제외

성능 개선: 문자 순회 기반 토크나이저

정규식 기반 RegexTokenizer를 문자 단위로 순회하는 SimpleTokenizer로 변경



44%

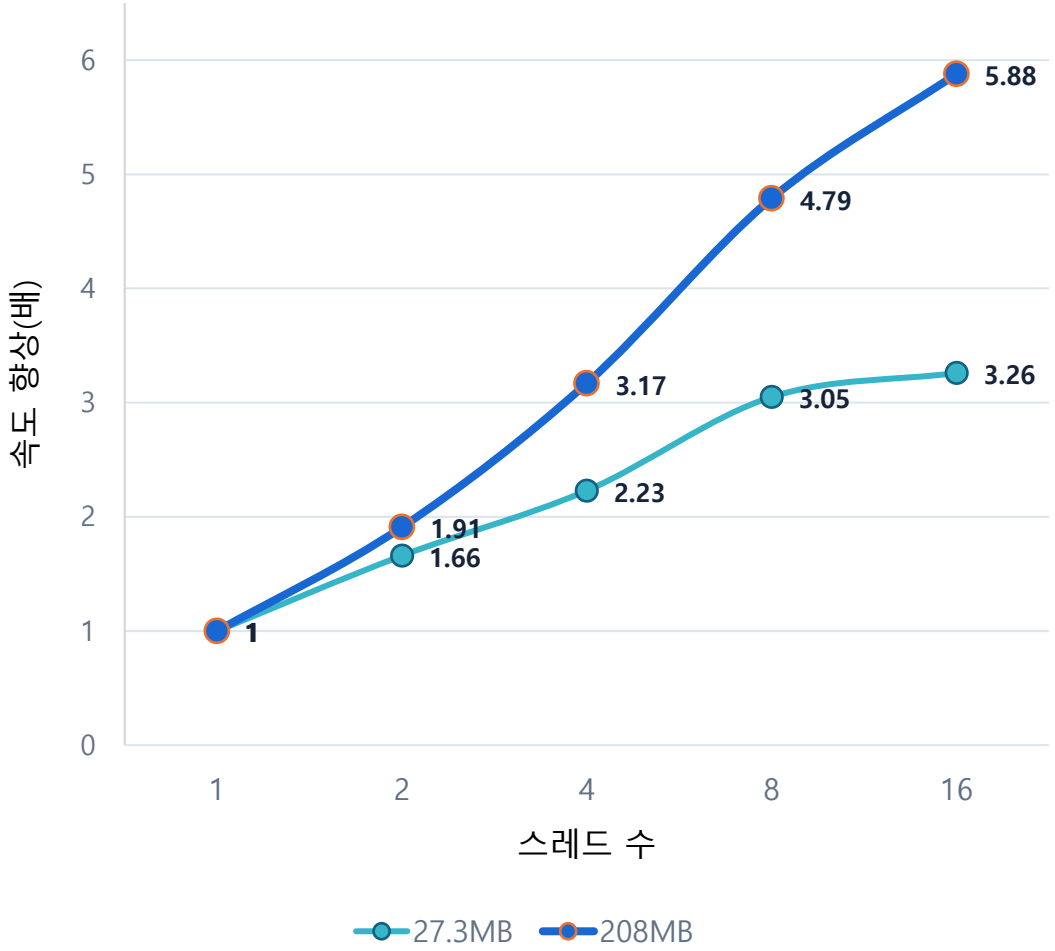
27.3MB 파일 처리 시간 감소

43%

208MB 파일 처리 시간 감소

병렬 처리

스레드 풀에서 파일별 단어 빈도를 병렬 집계한 뒤, 집계 결과를 순차 합산



5.88배

208MB 파일 16스레드

큰 입력에서 더 높은 병렬 성능 향상 확인

- 파일 읽기와 토큰화 비용의 비중이 커져 병렬 효율 증가

스레드 수에 비례하는 성능 향상에는 도달하지 못함

- 스레드 풀 생성, 작업 제출·관리와 결과 합산에 따른 고정 비용

병렬 처리 시연

같은 16개 파일을 1스레드와 4 스레드로 분석하여 처리 시간과 결과 비교

입력	**data/local/perf-64/many**
처리방식	**FIXED_THREAD_POOL**
스레드 수	**MAX_THREAD_COUNT: 1 → 4**

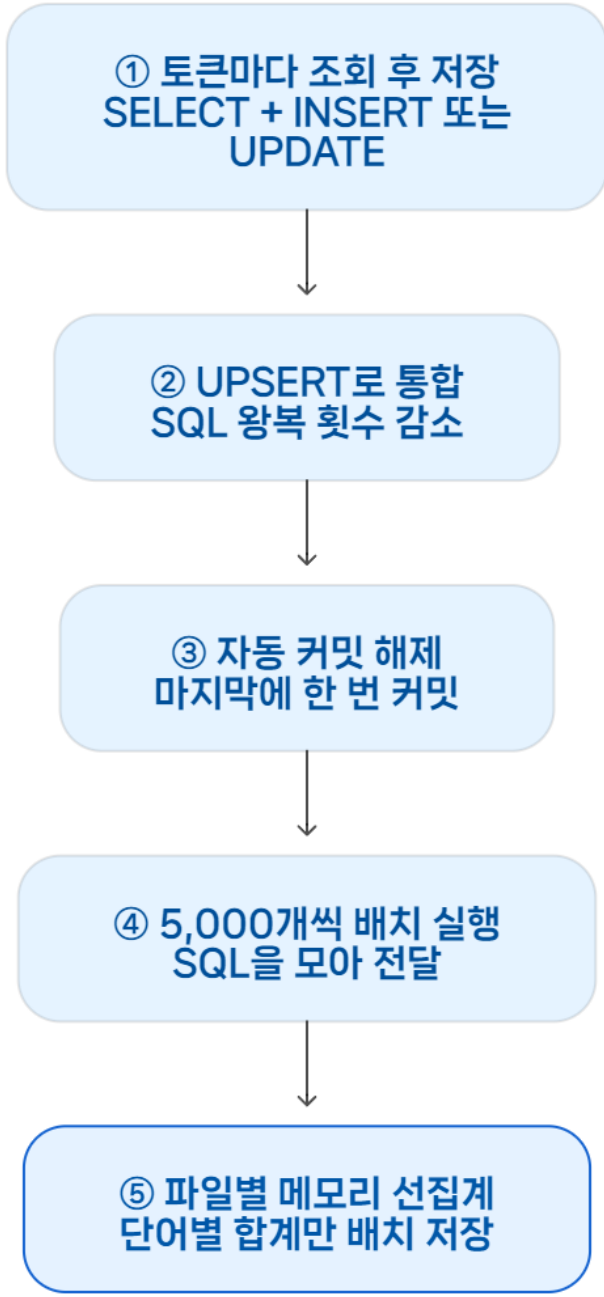
파일별 독립 집계로 병렬 처리하고, 성공한 결과만 순차 합산

DB 저장 방식 최적화

단어별 집계 결과를 메모리 대신 DB에 저장하고
저장 방식을 단계적으로 최적화

SQL 왕복·커밋 횟수와 저장할 데이터 수를

줄여 처리 시간 단축



체크포인트 저장과 중단 후 재개

파일별 집계 결과를 저장하고
재실행 시 유효한 체크포인트를 복원하여 중복 분석 방지

